

```
1. algoritmi:
2.
3. 1. dfs
4. complexitate:
5. a) lista de adiacenta:  $O(n + m)$ 
6. b) matrice de adiacenta:  $O(n^2)$ 
7.
8. 2. bfs
9. complexitate:
10. a) lista de adiacenta:  $O(n + m)$ 
11. b) matrice de adiacenta:  $O(n^2)$ 
12.
13. 3. prim: mst(apm) dintr un nod
14. complexitate:
15. a) heap:  $O(m \log n)$ 
16. b) vectori:  $O(n^2)$ 
17.
18. 4. kruskal
19. complexitate:
20. a) vectori de reprezentanti:  $O(m \log n + n^2)$ 
21. b) arbori Union/Find:  $O(m \log n)$ 
22.
23. 5. sortare topologica
24. complexitate:
25. a) DFS:  $O(n + m)$ 
26.
27. 6. distanta minima de la un nod la celelalte fara muchii negative: dijkstra
28. complexitate:
29. a) vectori:  $O(n^2)$ 
30. b) min-heap:  $O(m \log n)$ 
31. c) heap fibonacci:  $O(m + n \log n)$ 
32.
33. 7. distanta minima de la un nod la celelalte cu muchii negative fara cicluri: DAG
34. complexitate:  $O(n)$ 
35.
36. 8. distanta minima de la un nod la celelalte cu muchii negative fara cicluri negative:
    bellmanford
37. complexitate:
38. a) queue:  $O(n * m)$ 
39.
40. 9. maxflow
41. complexitate:
42. a) ford fulkerson:  $O(m * L\text{-taietura minima}) = O(n * m * C\text{-capacitate max})$ 
43. b) edmonds karp:  $O((n + m) * n * m) = O(n * m^2)$ 
44.
45. 10. roy floyd
46. complexitate:  $O(n^3)$ 
47.
48. 11. euler
49. complexitate:  $O(m)$ 
50.
51. 12. hamilton
52. complexitate:  $O(m * 2^n)$ 
53.
54. 13. maximum bipartite matching
55. complexitate:
56. a) curs:  $O(m * n)$ 
57. b) hopcroft karp:  $O(m \sqrt{n})$ 
58.
59.
60. Pseudocod:
```

```

61. 1. dfs
62. ...
63.
64.
65. 2. bfs
66. ...
67.
68.
69. 3. prim: mst(apm) dintr un nod
70. s - vf start
71. initial, T = ({s}, mult. vida)
72. pentru i = 1, n - 1
73.     alegem o muchie uv cu cost min din Gf a.i. u ap. V(T) si v nu ap. V(T)
74.     V(T) = V(T) + {v}
75.     E(T) = E(T) + uv
76.
77. Prim(G, w, s)
78.     pentru fiecare u ∈ V executa
79.         d[u] = ∞; tata[u] = 0
80.         d[s] = 0
81.         initializeaza Q cu V
82.         cat timp Q ≠ ∅ executa
83.             u = extrage varf cu eticheta minima din Q
84.             pentru fiecare v adiacent cu u executa
85.                 daca v ∈ Q si w(u, v) < d[v] atunci
86.                     d[v] = w(u, v)
87.                     tata[v] = u
88.             // actualizeaza Q - pentru Q heap
89. scrie (u, tata[u]), pentru u ≠ s
90.
91.
92. 4. kruskal
93. initial, T = (V, mult. vida)
94. pentru i = 1, n - 1
95.     alegem o muchie uv cu cost min din Gf a.i. sunt din componente conexe dif. (T + uv
        aciclic)
96.     E(T) = E(T) + uv
97.
98. sorteaza(E)
99. for (v=1; v <= n; v++)
100.     Initializare(v);
101.     nrmsel=0
102.     for (uv ∈ E)
103.         if (Reprez(u) != Reprez(v)) {
104.             E(T) = E(T) ∪ {uv};
105.             Reuneste(u,v);
106.             nrmsel = nrmsel + 1;
107.             if (nrmsel == n-1)
108.                 STOP; // break
109.         }
110.
111. void Initializare(int u) {
112.     tata[u] = h[u] = 0;
113. }
114. int Reprez(int u) {
115.     while (tata[u] != 0)
116.         u = tata[u];
117.     return u;
118. }
119.
120. void Reuneste(int u, int v) {

```

```

121.         int ru, rv;
122.         ru = Reprez(u);
123.         rv = Reprez(v);
124.         if (h[ru] > h[rv])
125.             tata[rv] = ru;
126.         else {
127.             tata[ru] = rv;
128.             if (h[ru] == h[rv])
129.                 h[rv] = h[rv] + 1;
130.         }
131.     }
132.
133.
134.     5.
135.     cât timp  $|V(G)| > 0$  execută
136.         alege v cu  $d^-(v) = 0$ 
137.         adauga v in ordonare
138.          $G \leftarrow G - v$ 
139.
140.
141.     6. dijkstra:
142.     Dijkstra(G, w, s)
143.         inițializează mulțimea vârfurilor neselectate Q cu V
144.         pentru fiecare  $u \in V$  execută
145.              $d[u] = \infty$ ;  $tata[u] = 0$ 
146.          $d[s] = 0$ 
147.         cât timp  $Q \neq \emptyset$  execută
148.             u = extrage vârf cu eticheta d minimă din Q
149.             pentru fiecare  $uv \in E$  execută
150.                 dacă  $d[u] + w(u, v) < d[v]$  atunci
151.                      $d[v] = d[u] + w(u, v)$ 
152.                      $tata[v] = u$ 
153.
154.     scrie d, tata
155.     // scrie drum minim de la s la un vârf t dat folosind tata
156.
157.
158.     8.
159.     pentru fiecare  $u \in V$  executa
160.          $d[u] = \text{infinit}$ ;  $tata[u] = 0$ 
161.      $d[s] = 0$ 
162.     pentru  $i = 1, |V| - 1$  executa
163.         pentru fiecare  $uv \in E$  executa
164.             dacă  $d[u] + w(u, v) < d[v]$  atunci
165.                  $d[v] = d[u] + w(u, v)$ 
166.                  $tata[v] = u$ 
167.
168.
169.     9.
170.     9.1 Algoritm generic de determinare a unui flux maxim
171.     Fie f un flux în N (de exemplu,  $f \equiv 0$  fluxul vid:  $f(e) = 0, \forall e \in E$ )
172.     Cât timp există un s-t lanț f-nesaturat P în G
173.         determinăm un astfel de lanț P
174.         revizuieste fluxul f de-a lungul lanțului P
175.     returneaza f
176.
177.     9.2 Schema:
178.     inițializează_flux_nul()
179.     cât timp (construieste_s-t_lanț_nesat_BF() == true) execută
180.         revizuieste_flux_lanț()
181.     afișează_flux()

```

```

182.
183.
184.     9.3 revizuiește_flux_lanț():
185.     fie P s-t lanțul găsit în construiește_s-t_lanț_nesat_BF()
186.     calculăm i(P)
187.     pentru fiecare arc e al lanțului P
188.         creștem cu i(P) fluxul pe e dacă este arc direct
189.         scădem cu i(P) fluxul pe e dacă este arc invers
190.
191.     9.4 construiește_s-t_lanț_nesat_BF()
192.     pentru v ∈ V execută
193.         tata[v] ← 0; viz[v] ← 0
194.     coada C ← ∅
195.     adaugă(s, C)
196.     viz[s] ← 1
197.     cât timp C ≠ ∅ execută
198.         i ← extrage(C)
199.         pentru ij ∈ E execută // arc direct
200.             dacă viz[j] = 0 și c(ij) > f(ij) atunci
201.                 adaugă(j, C)
202.                 viz[j] ← 1
203.                 tata[j] ← i
204.             dacă j = t atunci
205.                 STOP și returnează true (1)
206.         pentru ji ∈ E execută // arc invers
207.             dacă viz[j] = 0 și f(ji) > 0 atunci
208.                 adaugă(j, C)
209.                 viz[j] ← 1
210.                 tata[j] ← -i
211.             dacă j = t atunci
212.                 STOP și returnează true (1)
213.     returnează false (0)
214.
215.
216.
217.
218.     Teorema poliedrală a lui EULER:
219.     Fie G = (V, E) un graf planar conex și M = (V, E, F) o hartă a lui.
220.     Are loc relația:
221.          $|V| - |E| + |F| = 2$ 
222.
223.
224.     Flux:
225.     Fluxul este maxim - în mulțimea de arce evidențiată, toate arcele au
226.     flux=capacitate și nu putem construi drumuri de la s la t care nu conțin
227.     arce din această mulțime (s-t tăietură).
228.
229.     Fie N o rețea.
230.     O tăietură K se numește tăietură minimă în N dacă
231.      $c(K) = \min \{ c(K) \mid K \text{ este tăietură în } N \}$ 

```